Text Data Analysis & Visualization

Basic Text Plots, TF-IDF, Text Clustering and Classification

Xu Shirong

School of Economics, Xiamen University

2026

Outline

- Basic plots for text data
- TF-IDF and text representation
- Text clustering with K-Means
- Dimensionality reduction and visualization
- Interpreting and evaluating clusters

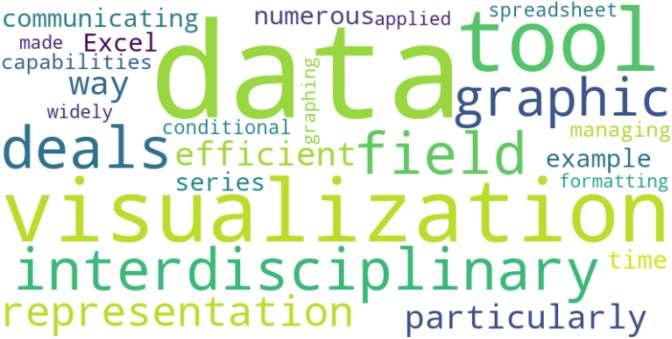
Basic Plots of Text

- Word Clouds: show frequent words, with word size reflecting frequency.
- Bar Charts: compare the frequencies of selected words or phrases.
- Bigram Networks: visualize common consecutive word pairs.
- Network Graphs: highlight relationships among words or entities.

Word Cloud Example

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text = "Data visualization is an interdisciplinary field that deals with the graphic
     representation of data. It is a particularly efficient way of communicating when
     the data is numerous as for example a time series. Excel's capabilities of
    managing spreadsheet data through data visualization tools, such as conditional
    formatting and graphing tools, have made it a widely applied data visualization
    tool."

wordcloud = WordCloud(
    width=800,
    height=400,
    background_color="white"
).generate(text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.show()
```

Result: Word Cloud



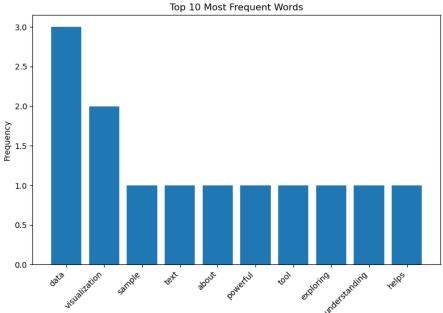
- "Data" is the most frequent word in this short example.
- Word clouds provide a quick overview of dominant terms.

Word Frequency Example

```
from collections import Counter

text = "This is a sample text about data visualization. Data visualization is a
    powerful tool for exploring and understanding data. It helps us see patterns and
    trends that might be difficult to identify from raw numbers."

text = text.lower()
text = "".join([char for char in text if char.isalnum() or char == " "])
words = text.split()

stopwords = ["a", "an", "the", "is", "of", "and", "to", "in",
             "on", "for", "it", "with", "as", "be", "have",
             "at", "by", "or", "that", "this", "from", "are"]

words = [word for word in words if word not in stopwords]
word_counts = Counter(words)
top_10_words = word_counts.most_common(10)

word_labels = [word for word, _ in top_10_words]
word_values = [count for _, count in top_10_words]
```

Bar Plot of Frequency

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
plt.bar(word_labels, word_values)
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.title("Top 10 Most Frequent Words")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
```



- "data" appears three times
- "visualization" appears twice
- Bar charts are more precise than word clouds

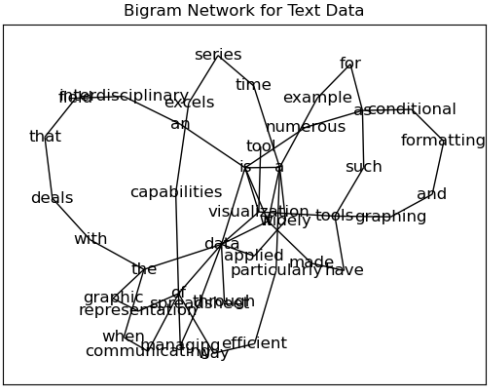
Procedure for Word Frequency Plots

- Step 1: convert the text to lowercase.
- Step 2: remove punctuation and stopwords.
- Step 3: count the frequencies of selected words.
- Step 4: visualize the top words with a bar chart.

Bigram Network Example

```python
import networkx as nx
from collections import Counter

processed_text = "".join(
    char.lower() for char in text
    if char.isalnum() or char.isspace()
)

words = processed_text.split()
bigrams = zip(words[:-1], words[1:])
bigram_counts = Counter(bigrams)

G = nx.Graph()
for bigram, count in bigram_counts.items():
    G.add_edge(bigram[0], bigram[1], weight=count)

pos = nx.spring_layout(G, seed=42)
nx.draw_networkx(G, pos, with_labels=True)
```

Result: Bigram Network



- Find common phrases
- Discover local relationships between words
- Useful for exploring themes in short texts
- Can become cluttered on very large corpora

Text Clustering Example: News Articles

- We analyze a subset of the 20 Newsgroups Dataset.
- Four categories are selected:
  - `rec.sport.hockey`
  - `talk.politics.misc`
  - `comp.graphics`
  - `sci.space`
- This is a standard example of unsupervised text mining.

Category Examples

comp.graphics

```
==========================================================================
Category: comp.graphics
--------------------------------------------------------------------------
I have a little question:

I need to convert RGB-coded (Red-Green-Blue) colors into HVS-coded
(Hue-Value-Saturnation) colors. Does anyone know which formulas to
use?

Thanks!
```

rec.sport.hockey

```
==========================================================================
Category: rec.sport.hockey
--------------------------------------------------------------------------


    They did last world cup and eventually lost tp Sweden in the final!
```

More Category Examples

talk.politics.misc

```
==========================================================================
Category: talk.politics.misc
--------------------------------------------------------------------------


       There are a couple of ways to look at them.  One is, "We want
you to support this Constitution, so we'll say anything that we think
will appeal to you," or the more straightforward, "This is why we think
what we've suggested in this Constitution is a good idea."


       You clearly consider the former to be the primary situation.
```

sci.space

```
==========================================================================
Category: sci.space
--------------------------------------------------------------------------


Sterrenkundig symposium 'Compacte Objecten'
                                          op 26 april 1993


In het jaar 1643, zeven jaar na de oprichting van de
Universiteit van Utrecht, benoemde de universiteit haar
eerste sterrenkundige waarnemer. Hiermee ontstond de tweede
universiteitssterrenwacht ter wereld. Aert Jansz, de eerste
waarnemer, en zijn opvolgers voerden de Utrechtse sterrenkunde
in de daaropvolgende jaren, decennia en eeuwen naar de
voorhoede van het astronomisch onderzoek. Dit jaar is het 350
jaar geleden dat deze historische benoeming plaatsvond.

De huidige generatie Utrechtse sterrenkundigen en studenten
sterrenkunde, verenigd in het Sterrekundig Instituut Utrecht,
vieren de benoeming van hun 'oervader' middels een breed scala
aan feestelijke activiteiten. Zo is er vo
```

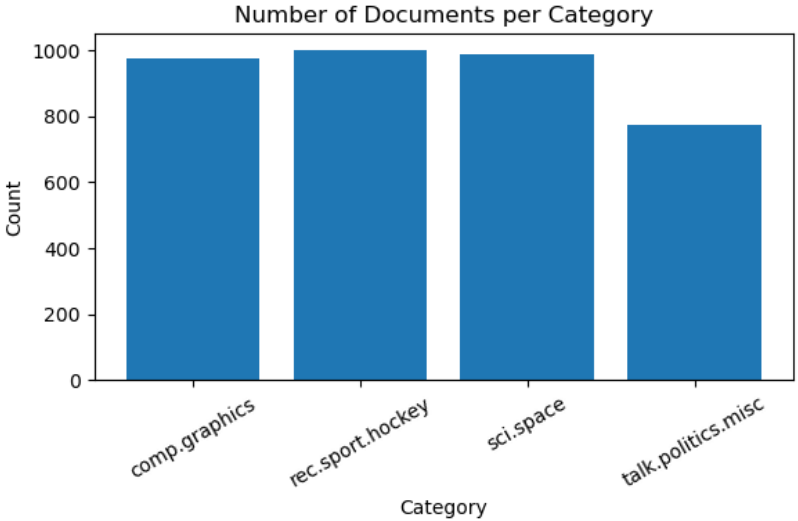
Workflow for Text Clustering

- Convert documents into numerical features with TF-IDF.
- Reduce dimensionality with LSA / Truncated SVD.
- Apply K-Means or MiniBatch K-Means.
- Visualize the result with PCA and t-SNE.
- Evaluate clusters with quantitative metrics.

Load Dataset

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import fetch_20newsgroups

categories = ["rec.sport.hockey","talk.politics.misc","comp.graphics","sci.space"]

dataset = fetch_20newsgroups(
    subset="all",
    categories=categories,
    remove=("headers", "footers", "quotes"),
    shuffle=True,
    random_state=42
)

labels = dataset.target
category_sizes = np.bincount(labels)
print(len(dataset.data))
```

Result: Category Distribution



What is TF-IDF?

- TF-IDF measures the importance of a term in a document relative to the full corpus.
- Common words such as “the” or “is” receive lower importance.
- Rare and topic-specific words receive higher importance.

Definition of TF-IDF

1. **Term Frequency (TF):**

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. **Inverse Document Frequency (IDF):**

$$\text{IDF}(t, D) = \log \frac{N}{1 + |\{d \in D : t \in d\}|}$$

3. **TF-IDF:**

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

TF-IDF Vectorization

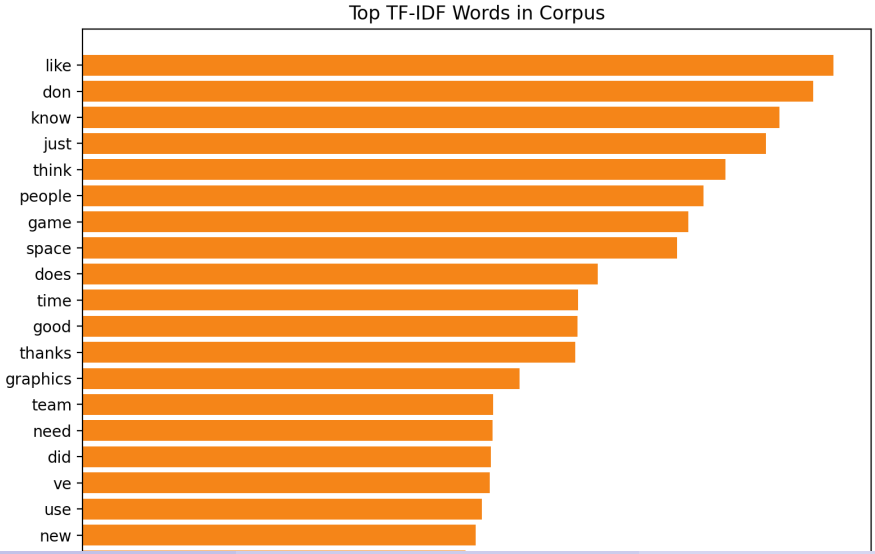
```python
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_df=0.5,
    stop_words="english"
)

X_tfidf = vectorizer.fit_transform(dataset.data)
terms = vectorizer.get_feature_names_out()

print(X_tfidf.shape)
```

- TF-IDF converts text into high-dimensional sparse vectors.
- Each column corresponds to one vocabulary term.

Top TF-IDF Words

```
tfidf_sum = np.asarray(X_tfidf.sum(axis=0)).ravel()
top_indices = tfidf_sum.argsort()[::-1][:20]

top_words = [terms[i] for i in top_indices]
top_scores = tfidf_sum[top_indices]

plt.figure(figsize=(7, 5))
plt.barh(top_words[::-1], top_scores[::-1])
plt.title("Top TF-IDF Words in Corpus")
```

Result: Top TF-IDF Words



Dimensionality Reduction: LSA

- Text data usually has very high dimensionality.
- TF-IDF vectors may contain thousands of features.
- We use Latent Semantic Analysis (LSA) to reduce dimension.
- LSA is implemented with:
  - Truncated SVD
  - normalization
- Benefits:
  - reduce noise
  - capture latent topics
  - speed up clustering

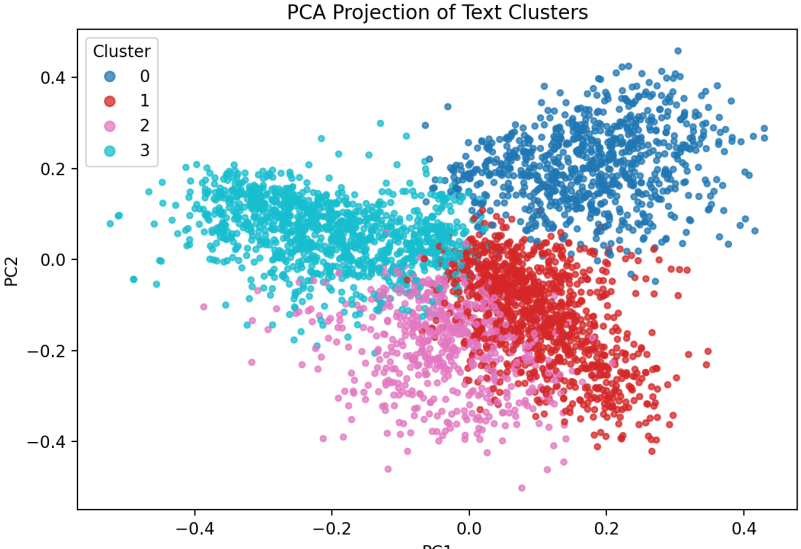
LSA Implementation

```
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

lsa = make_pipeline(
    TruncatedSVD(n_components=100, random_state=42),
    Normalizer(copy=False)
)

X_lsa = lsa.fit_transform(X_tfidf)
explained_var = lsa[0].explained_variance_ratio_.sum()
print(explained_var)
```

PCA Projection

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_lsa)

plt.figure(figsize=(6, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, s=10)
plt.title("PCA Projection of Documents")
```

Result: PCA Projection



Text Clustering

- After feature extraction we apply clustering.
- Two common algorithms are:
  - K-Means
  - MiniBatch K-Means
- MiniBatch K-Means is usually faster on large datasets.

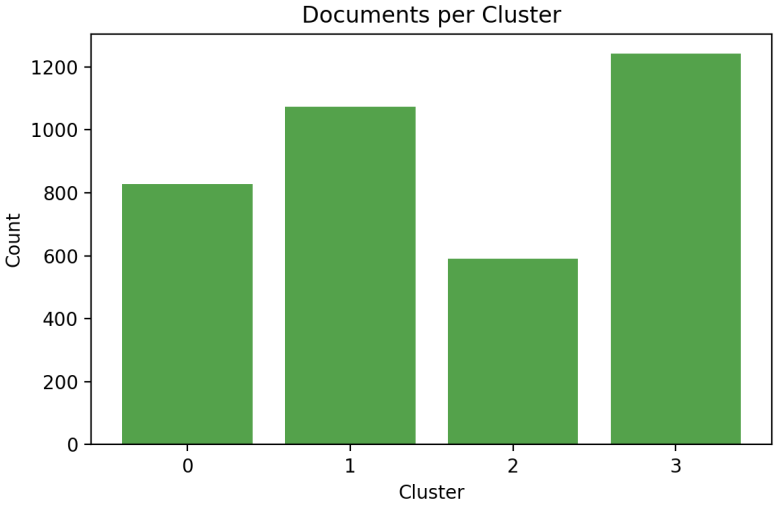
Clustering Model

```
from sklearn.cluster import KMeans

true_k = len(dataset.target_names)

kmeans = KMeans(
    n_clusters=true_k,
    max_iter=100,
    n_init=10,
    random_state=42
)

kmeans.fit(X_lsa)
```

Cluster Size Distribution

```
cluster_ids, cluster_sizes = np.unique(
    kmeans.labels_,
    return_counts=True
)

plt.bar(cluster_ids, cluster_sizes)
plt.title("Documents per Cluster")
plt.xlabel("Cluster")
plt.ylabel("Count")
```

Result: Cluster Sizes



Top Words per Cluster

```
original_space_centroids = lsa[0].inverse_transform(
    kmeans.cluster_centers_
)

order_centroids = original_space_centroids.argsort()[:, ::-1]

for i in range(true_k):
    print("Cluster %d:" % i)
    print([terms[ind] for ind in order_centroids[i, :10]])
```

- Important words help us interpret each cluster as a latent topic.

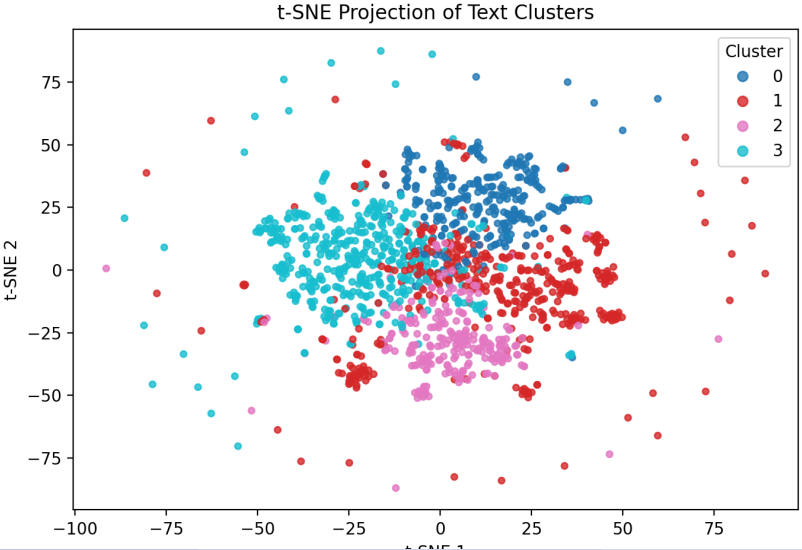
Cluster Interpretation

- Each cluster has representative words and a clear theme.
- Typical themes in this example may include:
  - hockey
  - politics
  - computer graphics
  - space science
- This makes unsupervised clustering easier to explain.

t-SNE Visualization

```
from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    perplexity=30,
    random_state=42
)

X_embedded = tsne.fit_transform(X_lsa)

plt.figure(figsize=(6, 5))
plt.scatter(
    X_embedded[:, 0],
    X_embedded[:, 1],
    c=kmeans.labels_,
    s=10,
    alpha=0.8
)
plt.title("t-SNE Projection of Text Clusters")
```

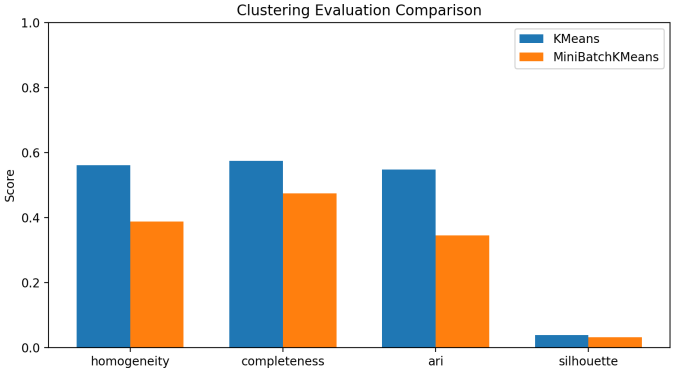
Result: t-SNE Visualization



Clustering Evaluation

- Homogeneity: each cluster contains documents from a single class.
- Completeness: documents from the same class are assigned to the same cluster.
- Adjusted Rand Index: similarity between predicted clusters and true labels.
- Silhouette Score: how well-separated the clusters are.

Result: Evaluation Comparison



- The figure compares clustering models across several metrics.
- It helps us balance interpretability, accuracy, and efficiency.

Summary

- Text data can be summarized with word clouds, bar charts, and bigram networks.
- TF-IDF transforms raw text into numerical features.
- LSA reduces dimensionality and improves clustering efficiency.
- K-Means groups similar documents into latent topics.
- PCA and t-SNE help visualize high-dimensional text data.